

ESCUELA DE INGENIERÍA INFORMÁTICA DE OVIEDO

Arquitectura de Computadores

Curso 2025-2026

Teamwork

Díaz Mendaña, Diego - UO301887

García Pernas, Pablo - UO300167

Rama García, Mateo - UO300710

Suárez Fernández, Fernando - UO300028

Grupo de prácticas: PL.1 - B

Titulación: PCEO Informática y Matemáticas

6 de octubre de 2025

Índice

1. Introducción	2
2. Desarrollo del algoritmo	2
2.1. Comprobaciones e inicialización	2
2.2. Implementación de algoritmo	4
3. Desarrollo del Proyecto	4
3.1. Fase 1: Implementación Monohilo	4
4. Resultados y Análisis	5
5. Conclusiones	5
 ANEXOS	 6
A. Flujo de trabajo y metodología de desarrollo	6
A.1. Metodología	6

1. Introducción

Esta memoria recoge el desarrollo del proyecto de la asignatura *Arquitectura de Computadores*, impartida en el curso 2025-2026 en la Escuela de Ingeniería Informática de Oviedo. El proyecto tiene como objetivo aplicar los conocimientos adquiridos en la asignatura para diseñar e implementar un filtro de imágenes en lenguaje C/C++. Además, se ha de analizar el rendimiento del filtro y compararlo con las implementaciones monohilo y multihilo.

El trabajo se ha estructurado en varias fases progresivas:

- **Entrega 1 (Monohilo):** Implementación básica del filtro de imágenes en un solo hilo.
- **Entrega 2 (SIMD y Multihilo):** Optimización del filtro utilizando instrucciones SIMD y paralelización con múltiples hilos.

2. Desarrollo del algoritmo

Se ha desarrollado el algoritmo de inversión de blanco y negro (Algoritmo número #3). Para desarrollar el código hemos usado la máquina virtual brindada por los profesores, así como la plantilla dada.

Dentro de esta plantilla ya nos venían incluidas las líneas de código con los que obteníamos tanto el número de píxeles como los punteros a los arrays RGB de la imagen destino y salida. Es por ello que nosotros únicamente hemos implementado las comprobaciones y el algoritmo.

2.1. Comprobaciones e inicialización

Antes de implementar el algoritmo, se debe realizar las comprobaciones necesarias para asegurarnos que la imagen existe y la inicialización de las variables. Para verificar que la imagen existe implementamos la siguiente función:

```
1 CImg<data_t> load_image(const char* filename) {  
2     if (access(filename, F_OK) == -1) {  
3         fprintf(stderr, "Error: la imagen '%s' no existe.\n", filename);  
4         exit(EXIT_FAILURE);  
5     }  
6  
7     CImg<data_t> image(filename);  
8     return image;  
9 }
```

Este método recibe en forma de parámetro el nombre del archivo y llama a la función `access()` con el nombre del archivo y el parámetro `F_OK`. Si la imagen no existe, la función `access()` devuelve `-1` y se muestra un mensaje de error por pantalla y se termina la ejecución del programa.

En caso de que la imagen exista, se crea un objeto de la clase `CImg` y se devuelve. También se ha de comprobar que las dimensiones de la imagen destino coinciden con las de la imagen original, así como el espectro. Esto se hace con el siguiente fragmento de código:

```
1  if (static_cast<unsigned int>(dstImage.width()) != width ||
2      static_cast<unsigned int>(dstImage.height()) != height ||
3      static_cast<unsigned int>(dstImage.spectrum()) != nComp) {
4      fprintf(stderr, "Error: las dimensiones de la imagen de salida no coinciden con la
        original.\n");
5      free(pDstImage);
6      exit(EXIT_FAILURE);
7  }
```

En caso de que las dimensiones no coincidan, se muestra una excepción por pantalla.

También modificamos el tipo de dato que se va a usar de float a double. Por otro lado definimos los pesos que se van a usar en el algoritmo y el brillo máximo. Como se muestra en el siguiente fragmento de código:

```
1  #define R_WEIGHT 0.3
2  #define G_WEIGHT 0.59
3  #define B_WEIGHT 0.11
4  #define MAX_BRIGHTNESS 255.0
```

Por otro lado, definimos las variables de tiempo.

```
1  struct timespec tStart, tEnd; // Variables for time measurement
2  double elapsedTime; // Elapsed time in seconds
```

Para medir el tiempo inicial y final usamos la función `clock_gettime()` con los parámetros `CLOCK_REALTIME` y la dirección de memoria de las variables `tStart` y `tEnd`. En caso de que la función falle, se muestra un mensaje de error y se termina la ejecución

```
1  if(clock_gettime(CLOCK_REALTIME, &tStart) == -1) {
2      printf("Error al obtener el tiempo inicial");
3      exit(EXIT_FAILURE);
4  }
5
6  if(clock_gettime(CLOCK_REALTIME, &tEnd) == -1){
7      printf("Error al obtener el tiempo final");
8      exit(EXIT_FAILURE);
9  }
```

Finalmente calculamos el tiempo transcurrido y lo mostramos por pantalla.

```
1  elapsedTime = (tEnd.tv_sec - tStart.tv_sec) + (tEnd.tv_nsec - tStart.tv_nsec) / 1e+9;
2  printf("Elapsed time (%d repetitions): %.6f seconds\n", N_ITERATIONS, elapsedTime);
```

2.2. Implementación de algoritmo

El algoritmo a implementar ha sido el de inversión de blanco y negro. Para ello implementamos la función `filter`, esta para cada pixel de la imagen multiplica cada componente del imagen original por su correspondiente peso, suma estos y lo atribuye a una variable `L` de tipo `double`. Los pesos son los siguientes:

1. Componente roja: Esta ha de ser multiplicada por 0.3
2. Componente verde: Esta ha de ser multiplicada por 0.59
3. Componente azul: Esta ha de ser multiplicada por 0.11

Por último hay que realizar la inversión de colores, es por ello hay que restar el valor obtenido a 255 (valor máximo de brillo) y asignar este valor a las tres componentes de cada pixel de la imagen destino. Como esto tiene que hacerse para cada pixel, realizamos un bucle que recorre todos los pixeles de la imagen. A continuación se muestra un fragmento de código de la función:

```
1 void filter (filter_args_t args) {  
2     for (uint i = 0; i < args.pixelCount; i++) {  
3         data_t L = *(args.pRsrc + i) * R_WEIGHT +  
4                 *(args.pGsrc + i) * G_WEIGHT +  
5                 *(args.pBsrc + i) * B_WEIGHT;  
6  
7         L = MAX_BRIGHTNESS - L;  
8  
9         *(args.pRdst + i) = L;  
10        *(args.pGdst + i) = L;  
11        *(args.pBdst + i) = L;  
12    }  
13 }
```

Como el tiempo debía estar entre 8 y 15 segundos, hemos realizado un bucle de 125 iteraciones de la función `filter` para que el tiempo de ejecución del programa esté dentro de este rango.

```
1 for (int i = 0; i < N_ITERATIONS; i++){  
2     filter(filter_args);  
3 }
```

A continuación se muestra la imagen original y la imagen resultante tras aplicar el filtro.

3. Desarrollo del Proyecto

3.1. Fase 1: Implementación Monohilo

Para la primera fase del proyecto, se ha desarrollado un algoritmo de



4. Resultados y Análisis

....

5. Conclusiones

....

ANEXOS

A. Flujo de trabajo y metodología de desarrollo

Todo el desarrollo del proyecto se puede seguir en el repositorio de *GitHub*: [ver repositorio](#). Para la gestión del proyecto y poder garantizar un desarrollo colaborativo, se ha establecido un flujo de trabajo común basado en *Git* y *GitHub* como herramientas centrales de organización y control de versiones. Los aspectos clave son:

- **Repositorio compartido:** con el código, documentación y recursos del proyecto.
- **Issue:** cada tarea se define como una *issue* en *GitHub*, con un responsable y un *deadline*.
- **Project Board:** se emplea un tablero *Kanban* con columnas *To Do*, *In Progress*, *Review* y *Done* para visualizar el estado de las tareas.
- **Milestones:** cada entrega del proyecto se ha asociado a un *milestone* que agrupa las *issues* correspondientes a dicha fase.
- **Pull Requests:** todas las contribuciones se realizan mediante *pull requests* que deben ser revisadas y aprobadas por, al menos, 3 de los 4 miembros del equipo antes de integrarse en *main*.
- **Documentación:** se guarda la documentación en la carpeta *docs/*.

A.1. Metodología

El equipo ha adoptado una metodología ágil adaptada al contexto académico del proyecto, inspirada en un enfoque híbrido entre *Scrum* y *Kanban*. Los aspectos clave de esta metodología son:

- La planificación se organiza en **iteraciones** (*milestones*) correspondientes a las entregas del proyecto.
- Las tareas se dividen en **issues** manejables y se priorizan según su importancia y dependencia. Estas tareas son asignadas a los miembros del equipo.
- El **project board** permite al equipo visualizar el progreso y gestionar las tareas de manera eficiente.
- La revisión colectiva de los *pull requests* fomenta la calidad del código y el aprendizaje compartido.
- Las dependencias entre issues se gestionan mediante etiquetas y referencias cruzadas en *GitHub*.